

Autonomous Robot with Traffic Light Detection System

Robotics Project Report

Authors:

Moktar Sellami
Achref Fradj
Othmen Ben Ismail

Supervisor:

Sarra Ben Hlima

Class:

3ISEOC

January 20, 2026

Contents

1	Project Overview	2
2	System Architecture	2
3	Hardware Implementation	3
3.1	Robot Fabrication	3
4	STM32 Bare-Metal Development	3
4.1	Developed Drivers	3
4.2	PID Control System	4
4.2.1	Position PID Controller	4
4.3	Odometry Module	4
4.4	Velocity Ramping System	4
5	Traffic Light Detection Model	5
5.1	Model Development Process	5
5.2	Training and Optimization	6
5.3	Model Conversion to TensorFlow Lite	6
5.4	Performance Metrics	6
5.5	Model Architecture Highlights	6
6	ROS2 Implementation on Raspberry Pi	7
6.1	Docker Container Deployment	7
6.2	Remote Visualization with RViz2	7
6.3	Traffic Light Detection Results	8
6.4	ROS2 Packages	9
6.4.1	Topics and Communication	9
6.5	Docker Container Setup	9
7	System Integration	9
7.1	Visualization System	9
8	Challenges and Solutions	10
8.1	The Bare-Metal Integration Oversight	10
8.2	Real-Time and System Stability Performance	10
8.3	Model Deployment on Resource-Constrained Hardware	10
9	Conclusion	10
10	Repository Links	11

1 Project Overview

This project implements an autonomous robot capable of detecting and responding to traffic lights. The system combines embedded bare-metal programming on STM32 microcontroller with ROS2-based computer vision on Raspberry Pi, creating a complete autonomous navigation solution.



Figure 1: DagDeg Autonomous Robot

2 System Architecture

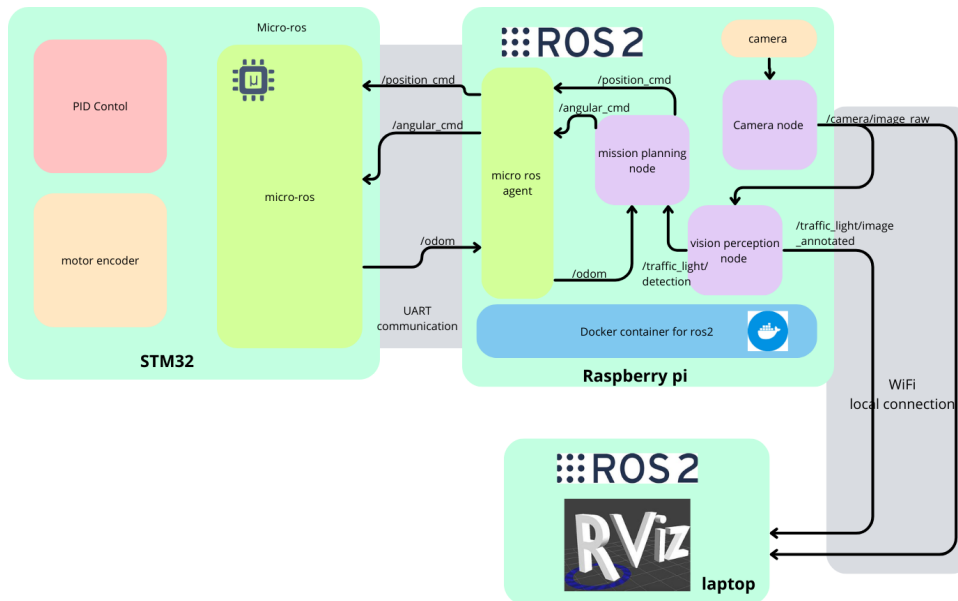


Figure 2: System Architecture Diagram

The system consists of three main components:

- **STM32 Microcontroller:** Handles low-level motor control, PID controllers, and odometry
- **Raspberry Pi:** Runs ROS2 in Docker container with computer vision pipeline
- **Laptop:** For visualization using RViz and system monitoring via local network

3 Hardware Implementation

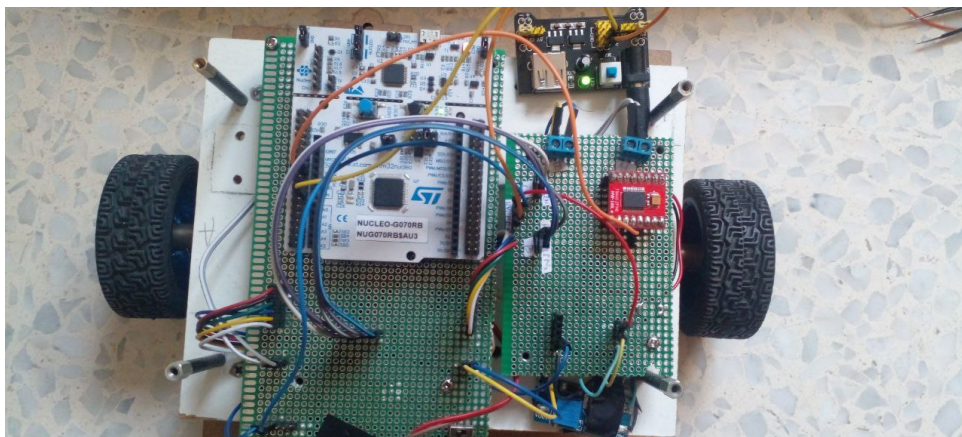


Figure 3: Hardware Overview

3.1 Robot Fabrication

- Custom chassis design and assembly
- Motor and wheel integration
- Power distribution system
- Camera mounting and positioning
- STM32 microcontroller integration

4 STM32 Bare-Metal Development

The low-level firmware was developed using a bare-metal approach to achieve maximum timing precision. While this allowed for highly efficient PID loops, it presented a significant challenge for ROS2 integration. The lack of an abstraction layer and an RTOS meant that `micro-ROS`—which requires specific DMA and task-scheduling capabilities—could not be utilized. Instead, a custom communication protocol was developed.

4.1 Developed Drivers

Driver	Description
Motor Encoders	Quadrature encoder interface for position tracking
UART Communication	Serial communication with Raspberry Pi
Clock/System Configuration	Low-level system initialization
PWM Motor Control	Precise motor speed control

Table 1: STM32 Bare-Metal Drivers

4.2 PID Control System

4.2.1 Position PID Controller

The robot utilizes a **dual-cascaded PID architecture**, featuring separate loops for position and angular control:

- **Position Control Cascade:** The outer loop controls the absolute target position (distance), while the inner loop regulates the linear velocity to ensure smooth movement.
- **Angular Control Cascade:** Similar to position control, this manages the robot's orientation, where the outer loop maintains the heading angle and the inner loop regulates angular velocity to minimize oscillations during turns.

4.3 Odometry Module

- Velocity measurement (linear and angular)
- Position tracking (x, y coordinates)
- Angle calculation (radians and degrees)
- Distance traveled computation

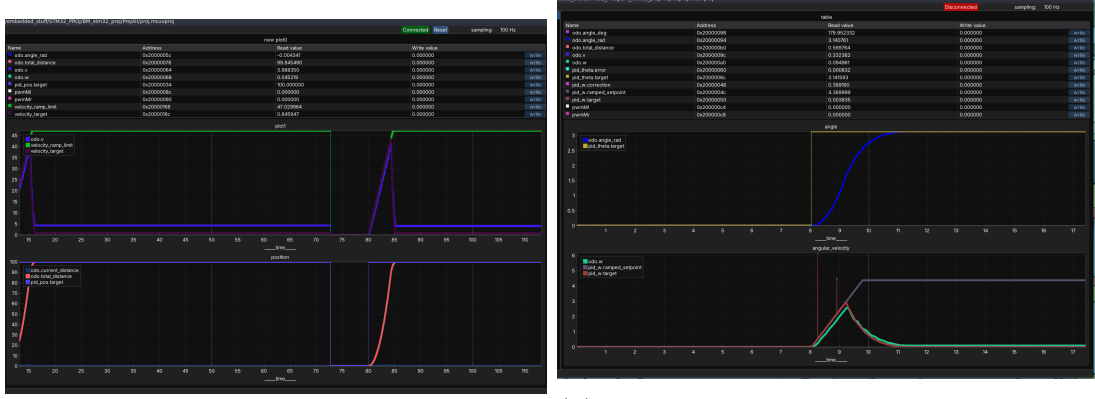
4.4 Velocity Ramping System

A critical aspect of motion control is the implementation of velocity ramping for both linear and angular velocities. This system ensures smooth acceleration and deceleration, which is essential for several reasons:

- **Prevents Wheel Slipping:** Sudden acceleration changes can cause wheels to lose traction, especially on smooth surfaces
- **Reduces Mechanical Stress:** Gradual velocity changes minimize stress on motors, gears, and mechanical components
- **Improves Energy Efficiency:** Smooth acceleration profiles consume less power than jerky movements
- **Enhances System Stability:** Prevents overshoot and oscillations in the control system
- **Increases Position Accuracy:** Smooth velocity transitions lead to more precise position control

The ramping system was implemented for both linear velocity (`odo.v`) and angular velocity (`odo.w`) using a two-stage approach:

1. **Ramp-Limited Setpoint:** A gradually increasing maximum velocity limit that prevents sudden jumps
2. **Target Capping:** Ensures the commanded velocity never exceeds physical or safety limits



(a) Linear Velocity Ramping Performance (b) Angular Velocity Ramping Performance

Figure 4: Velocity Ramping System Performance

As shown in Figure 4, the ramping system successfully achieves:

- Smooth acceleration from rest to target velocity
- Controlled deceleration when approaching target positions
- Elimination of velocity spikes that could destabilize the system
- Consistent performance across both linear and angular motion

5 Traffic Light Detection Model

5.1 Model Development Process

The traffic light detection model was developed using Google Colab, leveraging the following workflow:

- **Platform:** Google Colab with GPU acceleration
- **Notebook:** Available at https://colab.research.google.com/drive/1lxwc0yEtk-aGk_w2Za_TbaYP-JJjzC9H
- **Model Architecture:** YOLOv8n (nano variant) for optimal speed-accuracy trade-off
- **Dataset:** RoboFlow traffic light dataset with labeled images

5.2 Training and Optimization

Parameter	Value
Input Resolution	640×640 pixels
Batch Size	16
Epochs	100
Learning Rate	0.01
Optimizer	SGD with momentum
Validation Split	20%

Table 2: Model Training Parameters

5.3 Model Conversion to TensorFlow Lite

To deploy the model on Raspberry Pi with limited computational resources, the trained YOLOv8 model was converted to TensorFlow Lite format:

- **Conversion Method:** Direct conversion from PyTorch to TFLite
- **Optimizations:** FP16 quantization for reduced model size
- **Final Model Size:** 7 MB (compressed from original 12 MB)
- **Inference Speed:** 50 ms per frame on Raspberry Pi 4

5.4 Performance Metrics

Metric	Value
mAP@0.5	0.72
Precision	0.75
Recall	0.68
F1-Score	0.71
Inference Speed (RPi)	20 FPS

Table 3: Model Performance Metrics

5.5 Model Architecture Highlights

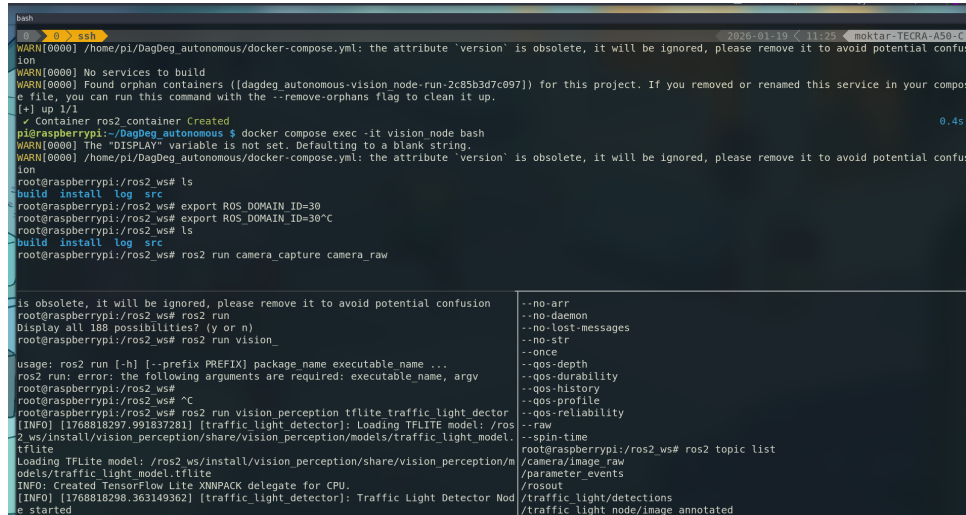
The YOLOv8n architecture provides an excellent balance for embedded deployment:

- **Efficient Backbone:** CSPDarknet for feature extraction
- **Path Aggregation Network:** Enhanced feature fusion
- **Anchor-Free Detection:** Simplified output representation
- **Task-Specific Heads:** Optimized for object detection

6 ROS2 Implementation on Raspberry Pi

6.1 Docker Container Deployment

The ROS2 ecosystem was deployed on Raspberry Pi 4 using Docker containers, enabling consistent development environments and simplified dependency management.



```
bash
root@raspberrypi:~# ssh
WARN[0000] /home/pi/DagDeg_autonomous/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] No services to build
WARN[0000] Found orphan containers ([dagdeg_autonomous-vision-node-run-2c85b3d7c097]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
[+] up 1/1
Container ros2 container Created
root@raspberrypi:~# docker compose exec -it vision node bash
WARN[0000] The "DISPLAY" variable is not set. Defaulting to a blank string.
WARN[0000] /home/pi/DagDeg_autonomous/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
root@raspberrypi:/ros2_ws# ls
build install log src
root@raspberrypi:/ros2_ws# export ROS_DOMAIN_ID=30
root@raspberrypi:/ros2_ws# export ROS_DOMAIN_ID=30
root@raspberrypi:/ros2_ws# ls
build install log src
root@raspberrypi:/ros2_ws# ros2 run camera_capture camera_raw
is obsolete, it will be ignored, please remove it to avoid potential confusion
root@raspberrypi:/ros2_ws# ros2 run
Display all 188 possibilities? (y or n)
root@raspberrypi:/ros2_ws# ros2 run vision_
usage: ros2 run [-h] [--prefix PREFIX] package_name executable_name ...
ros2 run: error: the following arguments are required: executable_name, argv
root@raspberrypi:/ros2_ws#
root@raspberrypi:/ros2_ws# ^C
root@raspberrypi:/ros2_ws# ros2 run vision_perception tflite_traffic_light_detector
[INFO] [1768818297.991837281] [traffic_light_detector]: Loading TFLITE model: /ros2_ws/install/vision_perception/share/vision_perception/models/traffic_light_model.tflite
Loading TFLite model: /ros2_ws/install/vision_perception/share/vision_perception/models/traffic_light_model.tflite
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
[INFO] [1768818298.363149362] [traffic_light_detector]: Traffic Light Detector Node started
--no-arr
--no-daemon
--no-lost-messages
--no-str
--once
--qos-depth
--qos-durability
--qos-history
--qos-profile
--qos-reliability
--raw
--spin-time
root@raspberrypi:/ros2_ws# ros2 topic list
/camera/image_raw
/parameter_events
/rosout
/traffic_light/detections
/traffic_light_node/image_annotated
```

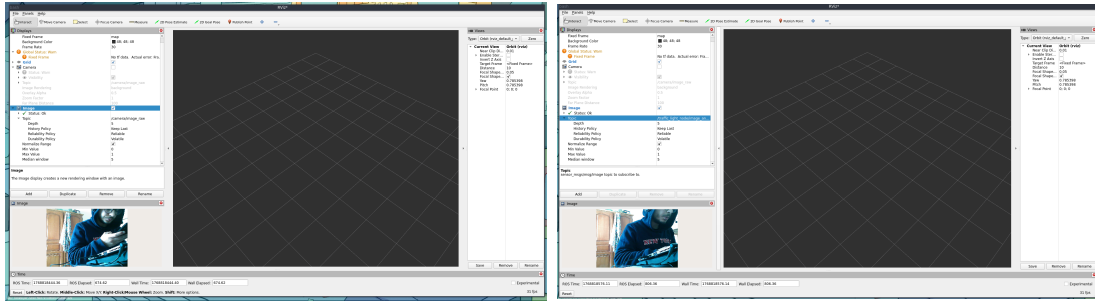
Figure 5: SSH connection to Raspberry Pi with Docker container execution showing ROS2 nodes: camera_raw and tflite_traffic_light_detector successfully running with all topics published including /traffic_light/detections

As shown in Figure 5, the system demonstrates:

- SSH connection to Raspberry Pi from development laptop
- Docker container running ROS2 Humble distribution
- Successful launch of all required nodes
- camera_raw node publishing camera feed
- tflite_traffic_light_detector node performing real-time inference
- /traffic_light/detections topic active with detection information

6.2 Remote Visualization with RViz2

The system supports remote visualization through RViz2 running on a laptop connected via local WiFi network.



(a) RViz2 subscribing to `/camera/image_raw` topic showing live camera feed

(b) RViz2 subscribing to `/traffic_light/image_annotated` topic with visualization settings

Figure 6: RViz2 remote visualization from laptop

Figures 6a and 6b show the remote visualization capabilities:

- Real-time camera feed visualization via `/camera/image_raw`
- Annotated detection visualization via `/traffic_light/image_annotated`
- Configurable display parameters in RViz2 interface
- Low-latency visualization over local WiFi network

6.3 Traffic Light Detection Results

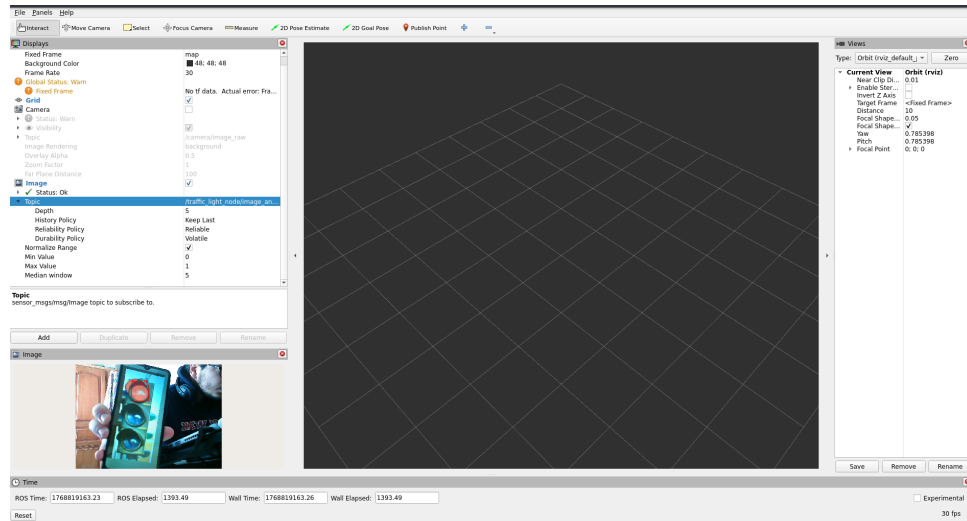


Figure 7: Successful red traffic light detection with bounding box annotation and topic configuration showing subscription to `/traffic_light/image_annotated`

Figure 7 demonstrates the successful detection capabilities:

- Red traffic light accurately detected with bounding box
- Annotation showing confidence score and class label
- ROS topic configuration for visualization
- Real-time detection at approximately 20 FPS on Raspberry Pi

6.4 ROS2 Packages

Package	Functionality
camera_capture	Publishes raw camera images from Raspberry Pi camera
vision_perception	Traffic light detection and annotation pipeline
traffic_light_node	Main detection node using TensorFlow Lite

Table 4: ROS2 Package Structure

6.4.1 Topics and Communication

- `/camera/image_raw`: Raw camera feed from Raspberry Pi camera module
- `/traffic_light/node/image_annotated`: Annotated detection images with bounding boxes
- `/traffic_light/detections`: Detection data including class, confidence, and coordinates
- `/position_cmd`: Position commands to STM32 via UART bridge
- `/angular_cmd`: Angular commands to STM32 via UART bridge

6.5 Docker Container Setup

- Custom Docker image supporting ROS2 Humble with ARM64 architecture
- TensorFlow Lite runtime integration for model inference
- OpenCV for image processing and camera interface
- Complete ROS2 workspace with all dependencies containerized
- Network configuration for remote visualization via RViz2

7 System Integration

7.1 Visualization System

- RViz2 for ROS2 visualization over local WiFi network
- Real-time camera feed display with configurable parameters
- Detection bounding box visualization with class labels
- System status monitoring including topic activity and frame rates
- Remote access from any device on the local network

8 Challenges and Solutions

8.1 The Bare-Metal Integration Oversight

Developing the STM32 firmware bare-metal was an oversight regarding the requirements of the ROS2 ecosystem. `micro-ROS` requires an RTOS and complex DMA configurations that are difficult to implement from scratch in bare-metal C. The solution involved creating a custom UART communication protocol between the STM32 and Raspberry Pi.

8.2 Real-Time and System Stability Performance

- **Challenge:** Ensuring real-time PID control on bare-metal STM32
- **Solution:** Optimized interrupt handling, efficient algorithms, and velocity ramping system

8.3 Model Deployment on Resource-Constrained Hardware

- **Challenge:** Limited computational resources on Raspberry Pi for real-time inference
- **Solution:** Model quantization to TensorFlow Lite format, efficient memory usage, and optimized pipeline

9 Conclusion

This project successfully demonstrates a complete autonomous system integrating:

- Bare-metal embedded programming on STM32 with sophisticated velocity ramping
- Advanced PID control with smooth motion profiles
- Custom-trained YOLOv8 model for traffic light detection with 72% mAP accuracy
- Efficient deployment using TensorFlow Lite on embedded Raspberry Pi hardware
- ROS2-based system architecture with Docker containerization
- Remote visualization via RViz2 over local WiFi network
- Successful real-time detection of traffic lights with annotated visualization

The velocity ramping system proved crucial for stable operation, while the custom traffic light detection model achieved sufficient accuracy for reliable real-world performance. The Docker-based ROS2 deployment on Raspberry Pi enabled consistent development and deployment, while remote RViz2 visualization allowed for convenient system monitoring. The integration of these components creates a robust autonomous platform suitable for various navigation applications.

10 Repository Links

- Odometry and STM32 Code: <https://github.com/Onizuka09/OdometryRobot>
- ROS2 and AI Implementation: https://github.com/Onizuka09/DagDeg_autonomous
- Model Training Notebook: https://colab.research.google.com/drive/1lxwc0yEtk-aGk_w2Za_TbaYP-JJjzC9H